



CENTRO UNIVERSITÁRIO DE BRASÍLIA — CEUB

Curso de Análise e Desenvolvimento de Sistemas / Ciência de Dados

Projeto Integrador — Análise de Requisitos

DOCUMENTO DE ARQUITETURA E DIAGRAMAS DE SOFTWARE

DAA

Projeto: Anahí (Projeto 3F)

Ipê Arc Studio · Versão 3.0 · 2026

EQUIPE TÉCNICA

Vítor Camargo da Silva Oliveira

Luís Eduardo Carvalho Ferreira

Rodrigo Seabra Vieira

Raphael Savini Bourrus Henriques

João Pedro de Melo Naves

HISTÓRICO DE REVISÕES

Versão	Data	Descrição	Autor
1.0	27/05/2026	Criação do documento de arquitetura: definição da arquitetura em camadas do jogo Anahí e do diagrama de sequência do núcleo de combate.	Equipe Técnica
3.0	02/06/2026	Alinhamento com o DAN v3.0 e o DDR v3.0: padronização de identidade visual (paleta institucional CEUB), nomenclatura técnica em inglês, rastreabilidade com funcionalidades (F1–F13), casos de uso (UC01–UC13) e regras de negócio (RN1–RN13).	Equipe Técnica

RESPONSÁVEIS PELA VALIDAÇÃO

Representante	Telefone / E-mail	Data / Assinatura
Representante do Cliente (Professora Lorena)		
Representante da Equipe Técnica (Raphael / Vítor)		
Representante Acadêmico (Direção Acadêmica CEUB)		

SUMÁRIO

1. INTRODUÇÃO4
 - 1.1 Objetivo do Documento4
 - 1.2 Escopo e Relação com o DAN e o DDR4
2. DIAGRAMA DE ARQUITETURA5
 - 2.1 Visão Geral da Arquitetura5
 - 2.2 Figura 1 — Arquitetura em Camadas6
 - 2.3 Descrição das Camadas7
 - 2.4 Decisões e Restrições Arquiteturais8
3. DIAGRAMA DE SEQUÊNCIA9
 - 3.1 Cenário Modelado (UC07 — Realizar Combate)9
 - 3.2 Figura 2 — Diagrama de Sequência do Combate10
 - 3.3 Objetos Participantes (Lifelines)11
 - 3.4 Descrição dos Passos da Sequência11
 - 3.5 Fluxos Alternativos e de Exceção Referenciados12

1. INTRODUÇÃO

1.1 Objetivo do Documento

Este Documento de Arquitetura e Diagramas de Software (DAA) tem por objetivo apresentar a visão arquitetural do jogo Anahí (Projeto 3F), complementando o Documento de Análise de Negócio (DAN) e o Documento de Definição de Requisitos (DDR). São apresentados dois artefatos: o Diagrama de Arquitetura, que organiza os componentes do sistema em camadas, e o Diagrama de Sequência, que descreve a interação dinâmica entre esses componentes no núcleo do gameplay.

Os dois diagramas mantêm rastreabilidade direta com os artefatos anteriores: cada componente e cada mensagem referencia as funcionalidades (F1–F13) definidas no DAN, os casos de uso (UC01–UC13), as regras de negócio (RN1–RN13) e as mensagens do sistema (M1–M26) especificados no DDR.

1.2 Escopo e Relação com o DAN e o DDR

O escopo cobre a aplicação como um jogo 2D Pixel Art do gênero Metroidvania / RPG, executado como aplicação desktop standalone, com persistência local em arquivo (sem servidor) e ambientação restrita a dois biomas brasileiros (Mata Atlântica e Caatinga), conforme as restrições registradas no DDR. A arquitetura proposta sustenta as 13 funcionalidades da Estrutura Analítica do Software (EAS) e os casos de uso priorizados via MoSCoW.

2. DIAGRAMA DE ARQUITETURA

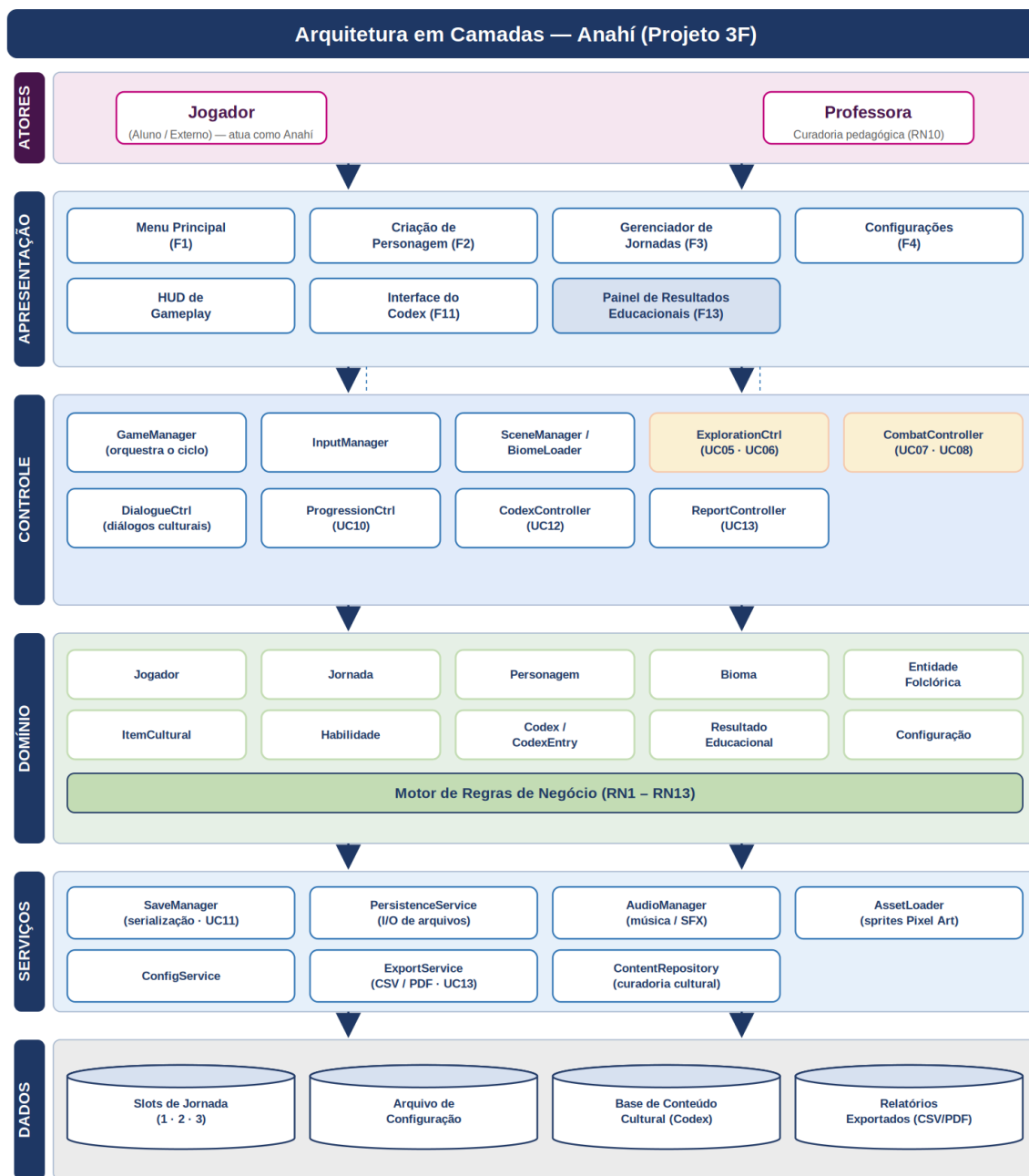
2.1 Visão Geral da Arquitetura

A arquitetura do Anahí adota o padrão em camadas (layered architecture), no qual cada camada depende apenas da camada imediatamente inferior. Essa organização separa a apresentação (interface e HUD), a lógica de controle do jogo (game loop e controladores), o modelo de domínio (entidades e regras de negócio), os serviços de infraestrutura e a camada de dados (persistência local em disco). Os atores Jogador e Professora interagem exclusivamente com a camada de apresentação.

A camada de Controle concentra o ciclo de jogo (GameManager) e os controladores que implementam os casos de uso de processamento e saída. A camada de Domínio reúne as entidades persistentes e transitórias derivadas do Diagrama de Classes do DDR, além do Motor de Regras de Negócio, que centraliza a aplicação das regras RN1 a RN13. A camada de Serviços isola operações de infraestrutura (serialização, I/O de arquivos, áudio, carregamento de recursos e exportação de relatórios).

2.2 Figura 1 — Arquitetura em Camadas

A Figura 1 apresenta a organização em camadas do sistema, com os atores no topo, o fluxo de dependência descendente (setas cheias) e a camada de dados na base.



Aplicação desktop standalone · 2D Pixel Art · persistência local em arquivo · sem servidor · paradigma Metroidvania / RPG

Figura 1 — Diagrama de Arquitetura em camadas do jogo Anahí (Projeto 3F). As setas cheias indicam a dependência descendente entre camadas; as setas tracejadas indicam respostas/retornos.

2.3 Descrição das Camadas

A tabela a seguir descreve a responsabilidade de cada camada e os principais componentes, com referência às funcionalidades (F), casos de uso (UC) e regras de negócio (RN) correspondentes.

Camada	Responsabilidade e principais componentes	Rastreabilidade
Atores	Jogador (atua como a protagonista Anahí) e Professora. Interagem somente com a camada de Apresentação.	DAN · Usuários
Apresentação (UI / HUD)	Telas e HUD: Menu Principal, Criação de Personagem, Gerenciador de Jornadas, Configurações, HUD de gameplay, Interface do Codex e Painel de Resultados Educacionais.	F1–F4 · F11 · F13
Controle (Game Loop)	GameManager, InputManager, SceneManager/BiomeLoader e controladores: Exploration, Combat, Dialogue, Progression, Codex e Report. Orquestram os casos de uso.	UC01–UC13
Domínio (Modelo + Regras)	Entidades: Jogador, Jornada, Personagem, Bioma, EntidadeFolclórica, ItemCultural, Habilidade, Codex/CodexEntry, ResultadoEducativo e Configuração. Inclui o Motor de Regras de Negócio.	DDR §2.3 · RN1–RN13
Serviços / Infraestrutura	SaveManager, PersistenceService, AudioManager, AssetLoader (Pixel Art), ConfigService, ExportService (CSV/PDF) e ContentRepository (curadoria cultural validada).	UC11 · UC13 · F12
Dados — Disco Local	Arquivos de jornada (slots 1–3), arquivo de configuração, base de conteúdo cultural do Codex e relatórios exportados.	RN9 · RN12

2.4 Decisões e Restrições Arquiteturais

As decisões arquiteturais a seguir derivam diretamente das restrições de projeto e das regras de negócio definidas no DAN e no DDR.

Decisão / Restrição	Justificativa	Origem
Aplicação desktop standalone, sem servidor	O jogo é single-player e o progresso é persistido localmente; não há necessidade de back-end em rede.	Escopo DDR
Persistência local em arquivo (máx. 3 slots)	Serialização do estado completo da jornada pelo SaveManager; suporte a no máximo 3 jornadas salvas.	RN9 · UC11
Renderização 2D Pixel Art, sem cutscenes	AssetLoader trabalha apenas com sprites/tilessets 2D; o jogador permanece no controle (paradigma Metroidvania).	RN4
Motor de Regras centralizado	Concentra cálculo de dano, chance de fuga e demais regras, evitando duplicação entre controladores.	RN5 · RN6
Acesso ao Painel restrito à Professora	ReportController e ExportService validam credenciais e modo pedagógico antes de gerar relatórios.	RN10 · UC13

3. DIAGRAMA DE SEQUÊNCIA

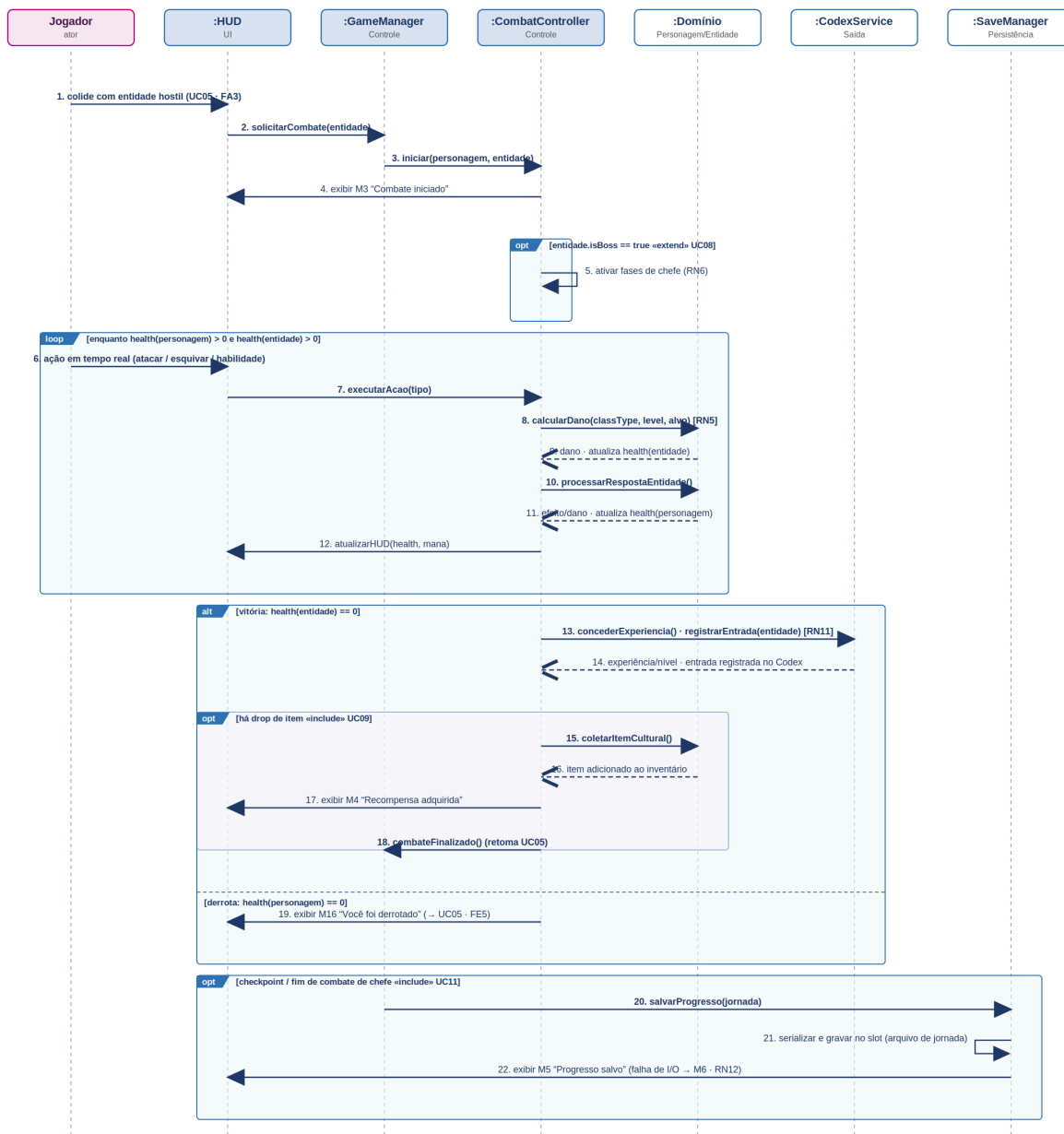
3.1 Cenário Modelado (UC07 — Realizar Combate)

O Diagrama de Sequência modela o núcleo do gameplay: a realização de um combate (UC07) disparado durante a exploração de um bioma (UC05). O cenário escolhido é representativo por exercitar a maior parte das camadas da arquitetura e por integrar três relacionamentos do Diagrama de Caso de Uso do DDR: o «extend» de UC08 (Enfrentar Chefe Folclórico) quando a entidade é um chefe, o «include» de UC09 (Coletar Item Cultural) ao final de um combate vitorioso com drop, e o «include» de UC11 (Salvar Progresso) ao atingir um checkpoint.

As mensagens trocadas entre os objetos referenciam as regras de negócio RN5 (cálculo de dano), RN6 (chance de fuga) e RN11/RN12 (registro no Codex e salvamento), bem como as mensagens do sistema M3, M4, M5, M6 e M16 definidas no DDR.

3.2 Figura 2 — Diagrama de Sequência do Combate

A Figura 2 apresenta as linhas de vida (lifelines) dos objetos participantes e a sequência de mensagens, com os fragmentos combinados opt (opcional), loop (repetição) e alt (alternativa).



Cenário: combate durante a exploração (UC05) — entidade comum, vitória com drop, salvamento em checkpoint. Setas cheias = chamada - setas tracejadas = retorno.

Figura 2 — Diagrama de Sequência do caso de uso UC07 (Realizar Combate), com os fragmentos opt, loop e alt. Setas cheias representam chamadas; setas tracejadas representam retornos.

3.3 Objetos Participantes (Lifelines)

Objeto (Lifeline)	Camada	Papel no cenário
Jogador	Ator	Aciona ações em tempo real (atacar, esquivar, usar habilidade).
:HUD	Apresentação	Captura entradas e exibe mensagens e indicadores (health, mana).
:GameManager	Controle	Orquestra o início/fim do combate e dispara o salvamento.
:CombatController	Controle	Conduz o loop de combate, aplica RN5/RN6 e concede recompensas.
:Domínio	Domínio	Personagem e EntidadeFolclórica: cálculo de dano, estado e drop de item.
:CodexService	Saída	Registra experiência/nível e entradas culturais no Codex (RN11).
:SaveManager	Persistência	Serializa e grava a jornada em disco (UC11 / RN12).

3.4 Descrição dos Passos da Sequência

A tabela detalha as mensagens numeradas da Figura 2, indicando o objeto de origem, o de destino e a referência ao DDR.

Nº	Origem → Destino	Mensagem	Ref.
1	Jogador → HUD	Colide com entidade folclórica hostil durante a exploração.	UC05 · FA3
2	HUD → GameManager	solicitarCombate(entidade).	UC07
3	GameManager → CombatController	iniciar(personagem, entidade).	UC07
4	CombatController → HUD	Exibe “Combate iniciado”.	M3
5	CombatController (self)	Se a entidade é chefe (isBoss), ativa fases de chefe; fuga indisponível.	UC08 · RN6
6	Jogador → HUD	Ação em tempo real: atacar, esquivar ou usar habilidade.	UC07
7	HUD → CombatController	executarAcao(tipo).	UC07
8	CombatController → Domínio	calcularDano(classType, level, alvo).	RN5
9	Domínio → CombatController	Retorna dano e atualiza health da entidade.	RN5
10	CombatController → Domínio	processarRespostaEntidade().	UC07

Nº	Origem → Destino	Mensagem	Ref.
11	Domínio → CombatController	Retorna efeito/dano e atualiza health do personagem.	UC07
12	CombatController → HUD	atualizarHUD(health, mana).	UC07
13	CombatController → CodexService	concederExperiencia() e registrarEntrada(entidade).	RN11
14	CodexService → CombatController	Experiência/nível atualizados; entrada registrada no Codex.	UC12
15	CombatController → Domínio	coletarItemCultural() (quando há drop).	UC09
16	Domínio → CombatController	Item adicionado ao inventário.	UC09
17	CombatController → HUD	Exibe “Recompensa adquirida”.	M4
18	CombatController → GameManager	combateFinalizado(); retoma a exploração.	UC05
19	CombatController → HUD	Em caso de derrota, exibe “Você foi derrotado”.	M16 · FE5
20	GameManager → SaveManager	salvarProgresso(jornada) em checkpoint / fim de chefe.	UC11
21	SaveManager (self)	Serializa e grava o estado no slot (arquivo de jornada).	RN12
22	SaveManager → HUD	Exibe “Progresso salvo” (em falha de I/O, exibe M6).	M5 · M6

3.5 Fluxos Alternativos e de Exceção Referenciados

Os fragmentos combinados da Figura 2 correspondem aos fluxos alternativos e de exceção especificados na seção 4 do DDR: o fragmento opt [isBoss] mapeia o «extend» UC08 (FA1 de UC07); o fragmento loop representa o combate em tempo real; o fragmento alt [vitória/derrota] cobre a concessão de recompensas e o FE5 de UC05 (derrota); o opt [drop] mapeia o «include» UC09; e o opt [checkpoint] mapeia o «include» UC11. As exceções de mana insuficiente (FE2/M17), tentativa de fuga contra chefe (FE6/M19) e falha de gravação (FE/ M6) permanecem especificadas no DDR e não foram incluídas na figura para preservar a legibilidade do cenário principal.

— Fim do Documento —

Documento de Arquitetura e Diagramas · Versão 3.0 · Ipê Arc Studio · 2026